Build Your Own AI System

A step-by-step, no-code manual — Predictive Maintenance on an IM-500 machine
LG Electronics Noida — AI Training take-home guide

> **What this is.** Everything you watched in the session, written so you can rebuild it at home for free. You'll first make an AI read your machine documents (RAG), then build three AI agents that work as a team, and finally make them log their decisions to a Google Sheet — with no coding.

1. What you'll build

One worked example, grown step by step:

1. **RAG** — an AI that answers only from your machine PDFs (NotebookLM).
2. **Three agents** that reason and hand off to each other (n8n):

Agent	Job	Input → Output
A — Failure Prediction	Predicts a machine failure from a sensor reading	sensor value → failing part + days to failure
B — Spare Parts	Checks the part's stock & lead time	failing part → stock, lead time, order decision
C — Maintenance	Raises the maintenance request	prediction + parts → scheduled request + priority

End-to-end: `RAG → your question → Agent A → Agent B → Agent C → Google Sheet`

2. Prerequisites (do once)

2.1 A Google account

Use a personal Gmail or your own domain — **not** a locked corporate account, which may block these tools. This one login covers NotebookLM, Gemini, and Google Sheets.

2.2 A free Gemini API key (needed for the agents, not for RAG)

1. Go to `aistudio.google.com` → **Get API key** → **Create API key**.
2. Copy it somewhere safe. This is what lets n8n talk to the AI model.

2.3 A free n8n Cloud account (for the agents)

1. Go to `n8n.io` → start a free cloud trial → sign in.
2. You land on your workflows home — this is where every agent lives.

3. Stage 1 — RAG: make the AI read YOUR machine docs (NotebookLM)

> **Why NotebookLM?** No-code, free, Google-stable. You upload PDFs and it answers *only* from them, with citations — no vector database to set up, nothing to manage. Best "just works" tool for grounded answers.

3.1 What RAG is (one line)

Give the AI your documents; it answers strictly from them and cites the source — instead of guessing from general knowledge.

3.2 Build it

1. Go to `notebooklm.google.com` → sign in with your Google account.
2. **Create new** notebook → name it "IM-500".
3. **Add sources** → upload the two machine PDFs: the *Failure Prediction Manual* and the *Spare Parts Catalog*.
4. Ask in the chat: `Servo position error is rising — which part, is it in stock, what's the risk?`

3.3 The before/after (the proof)

Step	What happens
Plain chatbot (ask the same question in `gemini.google.com`, no docs)	Vague or guessed — doesn't know your part numbers or stock. Not safe to act on.
NotebookLM, manuals only (no addendum yet)	Honestly says the servo encoder info is *not in the sources* — no hallucination.
NotebookLM + addendum (upload the EN-77 addendum PDF, ask again)	Full grounded answer: encoder EN-77, 0 in stock, 21-day lead, urgent — **cited** to the source.

> NotebookLM won't chat with zero sources. To show "before RAG," use plain Gemini (`gemini.google.com`) with no documents so the audience sees it guess.

4. n8n in 60 seconds

- **Workflow** — one automation on one canvas.
- **Node** — one step; wired left → right, data flows along the wires.
- **Trigger** — the first node (we use **Chat Trigger** = a live chat box).
- **AI Agent node** — the "brain"; you plug sub-nodes underneath it: a **Chat Model** (the LLM, required), optional **Memory**, **Tool**, and **Output Parser**.
- **Credential** — a saved secret (your API key / Google login), saved once and reused.

> The AI Agent no longer has an "agent type" setting (removed in n8n v1.82). Every AI Agent is a "Tools Agent" automatically.

> **Why n8n?** A no-code, visual builder with a free tier that "just works" in a browser — no servers, no licences. (Microsoft Copilot Studio needed a paid licence we didn't have; n8n needs none.)

5. Build Agent A (Failure Prediction)

1. Workflows home → **Create Workflow** → rename it "Agent A — Failure Prediction".
2. Click **+** → add **When chat message received** (Chat Trigger).
3. On the trigger, click **+** → search **AI Agent** → add it.
4. Under the AI Agent's **Chat Model** slot → **+** → **Google Gemini Chat Model**.
    - **Credential** → Create new → paste your Gemini API key → Save.
    - **Model** → pick a stable Flash model (see §9) — **not** a "preview" one.
5. Double-click the AI Agent → **Prompt** = "Take from previous node automatically".
6. Scroll to **Options** → **Add Option** → **System Message** → paste the Agent A prompt (Appendix A).
7. **Save** → **Open Chat** → test: `vibration 3.9 mm/s for 4 shifts`.

> Read the **chat bubble**, not the raw output panel — the raw panel shows escaped `\n` and `*`, which is normal.

6. Build Agent B (Spare Parts)

Same steps as Agent A, in a **new workflow**. Only differences:

- When adding the Gemini model, **reuse the same credential** from the dropdown.
- Paste the **Agent B** system message (Appendix A).

7. Build Agent C (Maintenance) + structured output

1. New workflow → build the agent exactly like B, paste the **Agent C** system message.

2. In the Agent C node, turn **ON** the toggle **"Require Specific Output Format"** (bottom of the Parameters tab). An **Output Parser** slot appears.
3. Under it → **+** → **Structured Output Parser** → Schema Type = **Generate From JSON Example** → paste:

```
{
  "machine": "IM-500 (Bay 3)",
  "component": "Servo Encoder EN-77",
  "part": "EN-77",
  "in_stock": 0,
  "failure_days": 6,
  "schedule_by_days": 6,
  "priority": "URGENT",
  "summary": "Raise URGENT maintenance; EN-77 out of stock, 21-day lead exceeds 6-day failure — escalate."
}
```

> **Keep the data clean.** In the Agent C prompt, force `priority` to be exactly `URGENT` or `NORMAL` (no notes), `part` to be the code only, and `component` to a fixed name list. Put any escalation note in `summary`. This keeps the logged Sheet clean and filterable.

8. Combine all three (orchestration)

A pipeline must live on **one canvas**. Don't rebuild — copy the agents in.

1. Open the Agent A workflow → click the AI Agent node, then **Ctrl-click** its Gemini model node (both selected) → **Ctrl+C**.
2. New workflow "Orchestration (A→B→C)" → **Ctrl+V**. Repeat for B and C.
3. Rename the three nodes: **Agent A**, **Agent B**, **Agent C**.
4. Add a Chat Trigger. Wire: **Chat Trigger** → **Agent A** → **Agent B** → **Agent C**.
5. Set each agent's **Prompt**:
   - **Agent A:** "Take from previous node automatically".
   - **Agent B:** "Define below" → Expression → `{{ $json.output }}`
   - **Agent C:** "Define below" → Expression →

```
Predicted failure: {{ $('Agent A').first().json.output }}
Parts status: {{ $('Agent B').first().json.output }}
```

> **Use `.first()`, not `.item`**, when referencing Agent A from Agent C — Agent A is two nodes back, and `.item` returns blank across two hops.

Test: Open Chat → `Servo position error is now 140 counts and rising` → A predicts EN-77 → B flags 0 stock / 21-day lead → C raises an URGENT request, escalate.

9. Choosing the model & free daily limits IMPORTANT

The free Gemini tier has a **per-model, per-day** request cap. Each orchestration run uses **3** requests (one per agent).

Model type	Free requests/day	Use it?
Preview (e.g. `gemini-3-flash-preview`)	~20 / day	No — you'll run out in ~6 runs
Stable Flash (e.g. `gemini-3.6-flash`)	~1,500 / day	Yes — ~500 runs/day

> **If you hit "429 Too Many Requests":** you exhausted *that model's* daily quota. The cap is **per model** — switch each agent's Chat Model to a *different* stable model (e.g. `gemini-3.6-flash`) and you get a fresh quota immediately. No new key or account needed.

> Model names change. If one says "no longer available", Google's error text usually names the replacement — use that. Then **lock it in** and stop switching (each model you touch spends its own quota).

10. Log decisions to a Google Sheet

> **Why a Sheet, not email?** A sheet is a permanent *record* you can filter, sort and share; email is only a one-off notification. The Sheet is the source of truth for every maintenance decision.

10.1 Create the sheet

1. `sheets.google.com` → blank → name it `IM500_Maintenance_Log`.
2. Row 1 headers (A1–H1):
   `Timestamp | Machine | Failing Component | Part No | In Stock | Failure In (days) | Schedule By (days) | Priority`

10.2 Add the Google Sheets node in n8n

1. In the orchestration workflow → add node → **Google Sheets**.
2. **Sign in with Google** → allow access.
3. Operation = **Append Row**; Document = `IM500_Maintenance_Log`; Sheet = `Sheet1`; Mapping = **Map Each Column Manually**.
4. Wire **Agent C** → **Google Sheets**.
5. Map each column (toggle each box to **Expression**):

```
Timestamp          {{ $now }}
Machine            {{ $json.output.machine }}
Failing Component  {{ $json.output.component }}
Part No            {{ $json.output.part }}
In Stock           {{ $json.output.in_stock }}
Failure In (days)  {{ $json.output.failure_days }}
Schedule By (days) {{ $json.output.schedule_by_days }}
Priority           {{ $json.output.priority }}
```

> If a value shows blank, check Agent C's Output tab — confirm the fields sit under `output`. If not, drop `.output` from the expressions. Run the chat → a new row appears in the Sheet. That's the agent taking a real action.

11. Troubleshooting

Symptom	Cause & fix
NotebookLM answers "not in sources"	Expected before you upload the relevant PDF — that's the honest, no-hallucination behaviour. Add the source and re-ask.
"No prompt specified / expected chatInput"	A downstream agent's Prompt is still "Take from previous node automatically". Set it to "Define below" + the expression.
Agent C shows "Not specified" for failure fields	Used `.item` across two hops. Use `$('Agent A').first().json.output`.
429 Too Many Requests	Daily quota for that model spent. Switch to a different stable model (§9).
Model 404 "no longer available"	Model retired. Use the replacement named in the error (e.g. `gemini-3.6-flash`).
Sheet shows messy values (e.g. `URGENT (escalate...)`)	Agent C output isn't clean. Force `priority` to exactly URGENT/NORMAL and `part` to the code only (§7); put notes in `summary`.
Output looks like `\n` and `*`	You're reading the raw panel. Read the chat bubble.

Appendix A — System prompts

Agent A — Failure Prediction

```
You are the Failure Prediction Agent for IM-500 injection molding (Bay 3). Predict
failures from sensor readings using ONLY this reference. If not covered, say so — don't
guess.

Thresholds (Normal / Warning / Critical -> cause, lead time):
- Vibration mm/s: <2.8 / 2.8-4.5 / >4.5 -> pump bearing wear, ~7d
- Oil temp C: <50 / 50-60 / >60 -> pump/cooling wear, ~10d
- Heater drift C: +/-5 / 5-10 / >10 -> heater band ageing, ~14d
- Motor current A: 18-24 / 24-28 / >28 -> overload, ~5d
- Screw torque %: <70 / 70-85 / >85 -> screw/barrel wear, ~12d
- Tie-bar strain uE: <800 / 800-1000 / >1000 -> tie-bar wear, ~9d
- Servo error counts: <50 / 50-120 / >120 -> encoder EN-77, ~6d

Rules: Warning = predicted failure; Critical = immediate. Schedule maintenance before
(today + lead time).
Output: failing component, lead time, urgency.
```

Agent B — Spare Parts

```
You are the Spare Parts Agent for IM-500 (Bay-3 store). Given a failing component/part,
return the exact part + stock using ONLY this reference. Don't invent parts or stock.

Component -> Part No (stock / min / lead / status):
- pump bearing -> BR-3105 (4/2/5d OK)
- pump seal -> SK-4620 (1/2/7d LOW)
- barrel heater -> HB-220 (3/2/6d OK)
- thermocouple -> TC-11 (0/1/4d OUT)
- cooling circuit -> CF-08 (6/3/3d OK)
- injection screw -> ST-90 (0/1/15d OUT)
- ejector -> EP-12 (2/1/5d OK)
- servo drive fan -> FN-40 (1/1/9d OK)
- servo encoder -> EN-77 (0/1/21d OUT)

Rules: stock < min -> reorder. If failure sooner than lead time -> URGENT/expedite.
EN-77 always URGENT (21d lead vs ~6d failure), escalate to Bay-3 lead.
Output: part no, stock, lead time, reorder/urgent decision.
```

Agent C — Maintenance (with clean fields)

```
You are the Maintenance Agent for IM-500 injection molding (Bay 3). You receive a
predicted failure and its spare-part status. Raise a maintenance request using ONLY
that input — don't invent details.

Keep two numbers separate: FAILURE lead time (days to failure) vs PART supply lead time
(days to get the part).
Rules:
- schedule_by_days must be LESS than failure_days (schedule before failure). Never use
the part supply lead time as the schedule date.
```

```
- If part out of stock OR part supply lead time > failure lead time -> priority URGENT,
note escalation in summary.
- Otherwise NORMAL.

Output these fields:
- machine: "IM-500 (Bay 3)"
- component: EXACTLY one of: Servo Encoder EN-77, Hydraulic Pump Bearing, Injection
Screw, Barrel Heater Band, Cooling Circuit, Hydraulic Pump Seal, Thermocouple
- part: part code ONLY (e.g. EN-77)
- in_stock: number only
- failure_days: number
- schedule_by_days: number, less than failure_days
- priority: EXACTLY "URGENT" or "NORMAL" — no notes
- summary: one-line request; put any escalation note HERE
```

Appendix B — Test inputs

Stage	Type this	Expect
RAG (NotebookLM)	Servo position error is rising — which part, is it in stock, what's the risk?	Before addendum: "not in sources". After: EN-77, 0 stock, 21d, urgent, cited.
Agent A	vibration 3.9 mm/s for 4 shifts	pump bearing, ~7d
Agent A (grounding)	What's the coolant pressure threshold?	refuses — not in reference
Agent B	Which parts are out of stock?	TC-11, ST-90, EN-77
Full chain	Servo position error is now 140 counts and rising	EN-77 → 0 stock, 21d → URGENT, escalate

Appendix C — Cheat sheet

- RAG needs only a Google login; agents need the Gemini key + n8n.
- Multi-select nodes = **Ctrl-click**.
- Only the first agent uses "Take from previous node automatically"; the rest use "Define below" + expression.
- Two hops back → `.first()`, not `.item`.
- Stable model, not preview. Lock it in once it works.
- Google Sheets: Append Row + Manual mapping.
- Keep Agent C's `priority` and `component` clean so the sheet stays tidy and filterable.